Sztuczna Inteligencja

Realizacja Sieci Neuronowej Uczonej
Algorytmem Wstecznej Propagacji Błędu
Metodą Momentum (MLP_M)

Imię i nazwisko: Dominik Starak

Spis treści

1 Cel projektu **2**

2 Część teoretyczna **2**
2.1 Model neuronu 2
2.2 Sieć neuronowa jednokierunkowa jednowarstwowa 4
2.3 Uczenie pod nadzorem sieci jednowarstwowej 5
2.4 Sieć neuronowa jednokierunkowa wielowarstwowa. 7
2.5 Uczenie pod nadzorem sieci wielowarstwowej 8
2.6 Przyśpieszanie procesu uczenia metodą momentum 10

3 Przygotowanie danych **10**

4 Skrypt programu **15**

5 Eksperymenty **23**
5.1 Sprawdzenie wpływu ilosci epok na dokładnosć modelu 23
5.2 Wyznaczanie optymalnych wartosci K1, K2, lr oraz mc 24

6 Podsumowanie i wnioski **26**

1 Cel projektu

Celem projektu było zbadanie wpływu różnych parametrów wejściowych na dokładność oraz wydajność algorytmu MLP (Multilayer Perceptron) uczącego się na danych medycznych pacjentów po zawale serca, z wykorzystaniem wstecznej propagacji błędu i przyśpieszenia metodą momentum. Zbadano wpływ na poprawność klasyfikacji następujących parametrów:

- Ilość neuronów 1 warstwy (K1)
- Ilość neuronów 2 warstwy (K2)
- Wartość przyspeszenia współczynnika uczenia (lr)
- Wartość współczynnika momentum (mc)

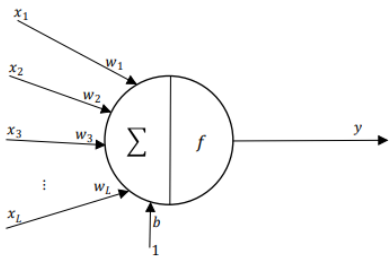
2 Część teoretyczna

2.1 Model neuronu

Sztuczny neuron w sztucznych sieciach neuronowych jest podstawową jednostką obliczeniową, której zadaniem jest przetwarzanie informacji. Składa się z trzech głównych elementów:

- Wejścia (inputs): Są to dane lub sygnały przekazywane do neuronu z innych źródeł.
- Wagi (weights): Parametry, które określają znaczenie poszczególnych wejść.
- Funkcja aktywująca: Matematyczna funkcja, która przekształca sumę ważoną wejść w sygnał wyjściowy neuronu.

Proces działania sztucznego neuronu polega na obliczeniu sumy ważonej wejść, a następnie zastosowaniu funkcji aktywacji do tej sumy, co generuje sygnał wyjściowy. Ten sygnał może być przekazywany do innych neuronów w sieci, umożliwiając rozwiązywanie złożonych problemów, takich jak klasyfikacja obrazów czy analiza danych tekstowych



Rysunek 1: Model neuronu.

Opis:
$x_1 - x_L$ - wartości wejściowe
$w_1 - w_L$ - wagi wejść
b - bias
Σ - sumator
f - funkcja aktywująca
y - wartość wyjściowa

Wartość wyjściową liczymy zależnością:

$$y = f\left(\sum_{j=1}^{L} w_j x_j + b\right) \tag{1}$$

Można użyć również zapisu macierzowego, gdzie:

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_L \end{bmatrix}, \quad w = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_L \end{bmatrix}$$

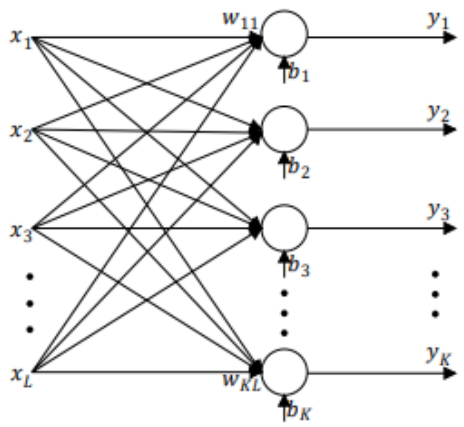
Wówczas wartość wyjściowa y jest wyrażona jako:

$$y = f(w \times x + b) \tag{2}$$

Natomiast ważona suma wejść z przesunięciem, oznaczona jako z, to:

$$z = \sum_{j=1}^{L} w_j \times x_j + b \tag{3}$$

2.2 Sieć neuronowa jednokierunkowa jednowarstwowa



Rysunek 2: Sieć jednokierunkowa jednowarstwowa

W sieci jednowarstwowej neurony są rozmieszczone w jednej warstwie. Wejściem dla neuronów jest wektor wartości wejściowych, przy czym każda wartość wejściowa jest połączona z każdym neuronem. Każdy neuron ma swój własny zestaw wag oraz bias.

Funkcjonowanie warstwy neuronów w zapisie macierzowym można określić za pomocą następującej formuły:

Wyrażenie

$$y = f(wx + b) \tag{4}$$

opisuje funkcjonowanie neuronu w warstwie sieci neuronowej. Oto składniki tego wyrażenia:

- x - wektor wejściowy, który zawiera dane wejściowe dla neuronu.
- w - wektor wag
- b - bias
- y - wynik działania neuronu

2.3 Uczenie pod nadzorem sieci jednowarstwowej

W trakcie procesu uczenia, w każdej epoce wagi przypisane do wartości wejściowych są dostosowywane, aby uzyskać jak najbardziej dokładne wyniki wyjściowe.
Wartość pojedynczej wagi neuronu jest określona przez wzór:

$$w_{ij}(t+1) = w_{ij}(t) + \Delta w_{ij}(t) \tag{5}$$

- t - numer epoki.

Nowe oszacowanie współczynnika wagowego wykorzystuje poprzednie oszacowanie powiększone o przyrost.

W uczeniu nadzorowanym każdemu wektorowi wejściowemu $\mathbf{x}$ przypisany jest wektor pożądanego wyjścia $\hat{\mathbf{y}}$, który jest wykorzystywany w procesie uczenia. Błąd niewytrenowanej sieci określany jest przez:

$$e = \mathbf{y} - \hat{\mathbf{y}} \tag{6}$$

Aby ujednolicić wartości wyjścia y oraz wartości oczekiwanej $\hat{y}$, można zminimalizować funkcję celu E, która najczęściej przyjmuje postać sumarycznego błędu kwadratowego (SSE).

$$E = \mathrm{SSE} = \frac{1}{2}\sum_{i=1}^{K} e_i^2 \tag{7}$$

Poszukiwanie minimum można przeprowadzić za pomocą metody gradientowej. Najczęściej stosowaną metodą gradientową do zmiany wag jest metoda największego spadku, opisana przez:

$$\Delta w = -\eta \nabla E(w) \tag{8}$$

- ∇ - symbol gradientu, który reprezentuje wektor pochodnych funkcji celu $E(w)$ względem wag w,
- η - współczynnik uczenia (learning rate), który kontroluje wielkość kroku w kierunku przeciwnym do gradientu,
- Δw - zmiana wag, która jest obliczana na podstawie gradientu funkcji celu,

- $E(w)$ - funkcja celu, która mierzy błąd lub różnicę między wyjściem sieci a oczekiwanym wynikiem.

Znak '-' w wzorze wynika z poszukiwania najszybszego spadku błędu, ponieważ gradient ∇E wskazuje kierunek najszybszego wzrostu funkcji celu.

Wyznaczając wzór uczenia gradientowego, ograniczając rozważania do i-tego neuronu i j-tej wagi, otrzymujemy:

$$\Delta w_{ij} = -\eta \frac{\partial E}{\partial w_{ij}} \tag{9}$$

Uwzględniając zależność $E = E(y_i(z_i(w_{ij})))$, pochodną możemy zapisać jako:

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial y_i} \cdot \frac{\partial y_i}{\partial z_i} \cdot \frac{\partial z_i}{\partial w_{ij}} \tag{10}$$

Uwzględniając poprzednie zależności (8) i (9), otrzymamy:

$$\frac{\partial E}{\partial y_i} = y_i - \hat{y}_i \tag{11}$$

A dzięki zależności (3):

$$\frac{\partial z_i}{\partial w_{ij}} = x_j \tag{12}$$

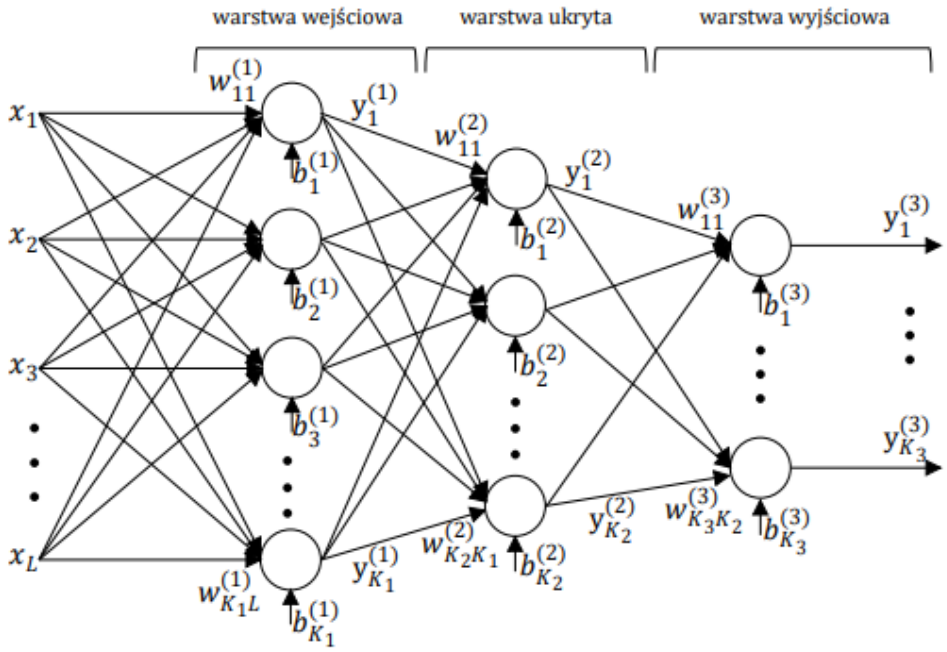
Podstawiając do (12) zależności (13) i (14), dostajemy regułę uczenia Widrowa-Hoffa:

$$\Delta w_{ij} = \eta \left(\hat{y}_i - y_i\right) \frac{\partial y_i}{\partial z_i} x_j \tag{13}$$

Przyjmując, że zależność $y = y(z)$ jest tangensoidalna, zależność (15) ostatecznie ma postać:

$$\Delta w_{ij} = \eta \left(\hat{y}_i - y_i\right) \left(1 - y_i^2\right) x_j \tag{14}$$

2.4 Sieć neuronowa jednokierunkowa wielowarstwowa.



Rysunek 3: Sieć jednokierunkowa wielowarstwowa.

W sieci wielowarstwowej warstwy neuronów są ułożone jedna po drugiej. Pierwszą warstwą jest warstwa wejściowa, ostatnią — warstwa wyjściowa, natomiast pomiędzy nimi znajdują się warstwy ukryte.

Każda warstwa posiada własne wektory wag, przesunięcia (biasy), wartości wyjściowe oraz funkcję aktywacji. Dla przejrzystości zapisu numer warstwy zapisuje się jako indeks dolny. Na przykład, dla warstwy trzeciej macierz wag oznaczona będzie symbolem $\mathbf{w}_2$.

Wyjście jednej warstwy stanowi wejście dla warstwy następnej. Dlatego macierz wag dla warstwy i ma wymiary $[K_i \times K_{i-1}]$, gdzie K^i to liczba neuronów w warstwie i, a K^{i-1} — w warstwie poprzedzającej.

Na przykład, druga warstwa posiada $L = K_2$ sygnałów wejściowych, $K = K_2$ neuronów oraz macierz wag $\mathbf{w}_2$ o rozmiarze $[K_2 \times K_1]$.

Wejściem do pierwszej warstwy jest wektor wartości wejściowych.

Działanie poszczególnych warstw opisane jest przez:

Działanie dla poszczególnych warstw określane jest przez:

$$\mathbf{y}^{(1)} = f^{(1)}(\mathbf{z}^{(1)}) = f^{(1)}(\mathbf{w}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) \tag{15}$$

$$\mathbf{y}^{(2)} = f^{(2)}(\mathbf{z}^{(2)}) = f^{(2)}(\mathbf{w}^{(2)}\mathbf{y}^{(1)} + \mathbf{b}^{(2)}) \tag{16}$$

$$\mathbf{y}^{(3)} = f^{(3)}(\mathbf{z}^{(3)}) = f^{(3)}(\mathbf{w}^{(3)}\mathbf{y}^{(2)} + \mathbf{b}^{(3)}) \tag{17}$$

Działanie całej sieci określone jest przez:

$$\mathbf{y}^{(3)} = f^{(3)}\left(\mathbf{w}^{(3)} f^{(2)}\left(\mathbf{w}^{(2)} f^{(1)}\left(\mathbf{w}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\right) + \mathbf{b}^{(2)}\right) + \mathbf{b}^{(3)}\right) \tag{18}$$

2.5 Uczenie pod nadzorem sieci wielowarstwowej

Dla sieci trójwarstwowej funkcja (7) przyjmie postać:

$$E = \mathrm{SSE} = \frac{1}{2}\sum_{i_3=1}^{K_3}(y_{i_3} - \hat{y}_{i_3})^2 \tag{19}$$

Po podstawieniu funkcji (18) otrzymamy:

$$E = \frac{1}{2}\sum_{i_3=1}^{K_3}\left(f^{(3)}\left(\sum_{i_2=1}^{K_2} w_{i_3 i_2}^{(3)} f^{(2)}\left(\sum_{i_1=1}^{K_1} w_{i_2 i_1}^{(2)} f^{(1)}\left(\sum_{j=1}^{L} w_{i_1 j}^{(1)} x_j + b_{i_1}^{(1)}\right) + b_{i_2}^{(2)}\right) + b_{i_3}^{(3)}\right) - \hat{y}_{i_3}\right)^2 \tag{20}$$

- gdzie $j = 1, \ldots, L$ - numer wyjścia warstwy pierwszej
- gdzie $i_1 = 1, \ldots, K_1$ - numer wyjścia warstwy drugiej
- gdzie $i_2 = 1, \ldots, K_2$ - numer wyjścia warstwy trzeciej
- gdzie $i_3 = 1, \ldots, K_3$ - numer wyjścia warstwy czwartej

Składniki gradientu funkcji poszczególnych warstw otrzymuje się przez różniczkowanie zależności (20). Obliczenia należy rozpocząć od warstwy wyjściowej.

$$\frac{\partial E}{\partial w_{i_3 i_2}^{(3)}} = \frac{\partial E}{\partial y_{i_3}} \cdot \frac{\partial y_{i_3}^{(3)}(z_{i_3}^{(3)})}{\partial z_{i_3}^{(3)}} \cdot \frac{\partial z_{i_3}^{(3)}}{\partial w_{i_3 i_2}^{(3)}} \tag{21}$$

Przekształcając analogicznie jak w (10)–(12) otrzymamy:

$$\frac{\partial E}{\partial w_{i_3 i_2}^{(3)}} = (y_{i_3}^{(3)} - \hat{y}_{i_3}) \cdot \frac{\partial y_{i_3}^{(3)}}{\partial z_{i_3}^{(3)}} \cdot y_{i_2}^{(2)} \tag{22}$$

Podobnie dla warstwy ukrytej:

$$\begin{aligned}\frac{\partial E}{\partial w_{i_2 i_1}^{(2)}} =& \frac{\partial E}{\partial y_{i_3}^{(3)}} \cdot \frac{\partial y_{i_3}^{(3)}(z_{i_3}^{(3)})}{\partial z_{i_3}^{(3)}} \cdot \frac{\partial z_{i_3}^{(3)}}{\partial y_{i_2}^{(2)}} \cdot \frac{\partial y_{i_2}^{(2)}(z_{i_2}^{(2)})}{\partial z_{i_2}^{(2)}} \\ & \cdot \frac{\partial z_{i_2}^{(2)}}{\partial w_{i_2 i_1}^{(2)}} = \sum_{i_3=1}^{K_3} (y_{i_3}^{(3)} - \hat{y}_{i_3}) \cdot \frac{\partial y_{i_3}^{(3)}(z_{i_3}^{(3)})}{\partial z_{i_3}^{(3)}} \cdot w_{i_3 i_2}^{(3)} \\ & \cdot \frac{\partial y_{i_2}^{(2)}(z_{i_2}^{(2)})}{\partial z_{i_2}^{(2)}} \cdot y_{i_1}^{(1)}\end{aligned} \tag{23}$$

Oraz wejściowej:

$$\begin{aligned}\frac{\partial E}{\partial w_{i_1 j}^{(1)}} =& \frac{\partial E}{\partial y_{i_3}^{(3)}} \cdot \frac{\partial y_{i_3}^{(3)}(z_{i_3}^{(3)})}{\partial z_{i_3}^{(3)}} \cdot \frac{\partial z_{i_3}^{(3)}}{\partial y_{i_2}^{(2)}} \cdot \frac{\partial y_{i_2}^{(2)}(z_{i_2}^{(2)})}{\partial z_{i_2}^{(2)}} \cdot \frac{\partial z_{i_2}^{(2)}}{\partial w_{i_2 i_1}^{(2)}} \\ & \cdot \frac{\partial y_{i_1}^{(1)}(z_{i_1}^{(1)})}{\partial z_{i_1}^{(1)}} \cdot \frac{\partial z_{i_1}^{(1)}}{\partial w_{i_1 j}^{(1)}} \\ & = \sum_{i_3=1}^{K_3} (y_{i_3}^{(3)} - \hat{y}_{i_3}) \cdot \frac{\partial y_{i_3}^{(3)}(z_{i_3}^{(3)})}{\partial z_{i_3}^{(3)}} \cdot \sum_{i_2=1}^{K_2} w_{i_3 i_2}^{(3)} \\ & \cdot \frac{\partial y_{i_2}^{(2)}(z_{i_2}^{(2)})}{\partial z_{i_2}^{(2)}} \cdot w_{i_2 i_1}^{(2)} \cdot \frac{\partial y_{i_1}^{(1)}(z_{i_1}^{(1)})}{\partial z_{i_1}^{(1)}} \cdot x_j\end{aligned} \tag{24}$$

2.6 Przyśpieszanie procesu uczenia metodą momentum

Metoda momentum jest popularną techniką przyspieszania procesu uczenia. Polega na wprowadzeniu do wzoru aktualizacji wag dodatkowego składnika, który uwzględnia bezwładność wag – zwanego momentem. W przypadku uczenia nadzorowanego, reguła aktualizacji wag przy użyciu gradientu, z zastosowaniem metody momentum, przyjmuje następującą postać:

$$w_{ij}(t+1) = w_{ij}(t) + \Delta w_{ij}(t) + m_c \cdot M_{ij}(t) \tag{25}$$

gdzie:

- m_c – współczynnik momentum,
- $M_{ij}(t)$ – składnik momentum.

Składnik momentum $M_{ij}(t)$ jest wyliczany na podstawie wzoru:

$$M_{ij}(t) = w_{ij}(t) - w_{ij}(t-1) \tag{26}$$

Współczynnik m_c należy do przedziału $[0, 1]$. Jeśli $m_c = 0$, reguła (25) upraszcza się do reguły (5). Najczęściej przyjmuje się wartość $m_c = 0.95$.

3 Przygotowanie danych

Zbiór danych wzięty został z https://archive.ics.uci.edu/dataset/38/echocardiogram, na zbiór składa się 132 rekordów, każdy rekord z 13 atrybutów. Wraz ze zbiorem pobrany był też plik z informacjami o nim, tak zostały opisane poszczególne atrybuty:

- **survival (przeżycie):** Liczba miesięcy, przez które pacjent przeżył (albo nadal żyje). Ponieważ pacjenci mieli zawały w różnych okresach, możliwe jest, że niektórzy pacjenci przeżyli mniej niż rok, ale nadal żyją. Należy sprawdzić drugi atrybut, aby to potwierdzić. Pacjenci, którzy przeżyli mniej niż rok, ale nadal żyją, nie mogą być używani w zadaniu predykcji (czy pacjent przeżyje rok).
- **still-alive (żyje nadal):** Zmienna binarna, gdzie:
  - 0 oznacza, że pacjent zmarł na końcu okresu przeżycia.
  - 1 oznacza, że pacjent żyje.
- **age-at-heart-attack (wiek w momencie zawału):** Wiek pacjenta w latach, kiedy doszło do zawału serca.

- **pericardial-effusion (wysięk osierdziowy):** Zmienna binarna dotycząca obecności płynu w okolicach serca:
  - 0 oznacza brak płynu.
  - 1 oznacza obecność płynu.
- **fractional-shortening (skrócenie frakcyjne):** Miarą kurczliwości serca. Niższe wartości oznaczają coraz bardziej nienormalną kurczliwość.
- **epss (separacja punktu E):** Miara kurczliwości serca. Większe wartości oznaczają coraz większe problemy z kurczliwością serca.
- **lvdd (wymiar koniuszka lewej komory podczas rozkurczu):** Miara wielkości serca podczas rozkurczu. Większe serca są zazwyczaj objawem choroby.
- **wall-motion-score (skala ruchów ścian):** Miara tego, jak segmenty lewej komory serca poruszają się.
- **wall-motion-index (indeks ruchów ścian):** Jest to wynik podzielony przez liczbę segmentów, które zostały zaobserwowane w echokardiogramie (zwykle jest ich 12-13). Zmienna ta powinna być używana zamiast wskaźnika ruchów ścian.
- **mult (nieistotna zmienna):** Zmienna pochodna, której nie należy brać pod uwagę.
- **name (imię pacjenta):** Nazwa pacjenta. Zmienna ta została zastąpiona jako „name” w zbiorze danych.
- **group (grupa):** Zmienna bez znaczenia, którą należy zignorować.
- **alive-at-1 (życie po roku):** Zmienna typu binarnego, która określa, czy pacjent żył po upływie roku:
  - 0 oznacza, że pacjent zmarł po roku lub nie był obserwowany przez rok.
  - 1 oznacza, że pacjent żył po roku.

Niektóre rekordy zawierały brakujące danych, są to również dane nieuporządkowane. W celu pozbycia się nich, zostały one usunięte poniższym skryptem. Otwiera on plik echocardiogram.data w trybie tylko do odczytu, otwiera plik wyjciowy w trybie zapisu, po czym przechodzi przez każdy rekord, jeśli

nie ma w nim znaku '?', zapisuje go do pliku wyjściowego. Następnie dane zostały przeniesione do pliku echocardiogram.data

```
with open('echocardiogram.data', 'r') as input_file:
    with open('filtered_data.txt', 'w') as output_file:
        for line in input_file:
            if '?' not in line:
                output_file.write(line)
```

Listing 1: Kod usuwający niepełne rekordy

Przygotowany plik echocardiogram.data zostaje potem przetworzony przez następny skrypt.

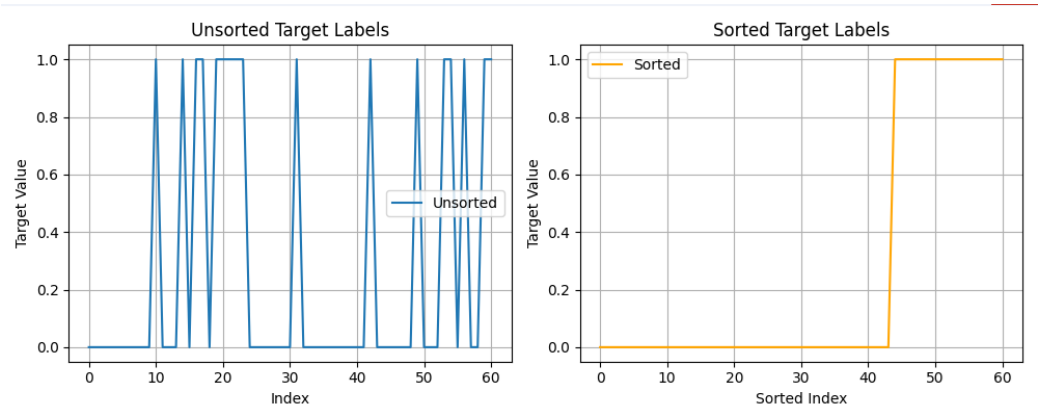
- Wczytanie danych z pliku `echocardiogram.data` (10)
- Oddzielenie cech i wartości docelowych (13)
- Obliczenie minimalnych i maksymalnych wartości cech (17)
- Normalizacja danych do zakresu [-1, 1] (23)
- Wypisanie zakresów po normalizacji (26)
- Sortowanie danych według wartości docelowych (29)
- Tworzenie wykresu dla danych nieposortowanych (36)
- Tworzenie wykresu dla danych posortowanych rosnąco (45)
- Zapis danych do pliku `echocardiogram.hkl` (56)

```
1 import hickle as hkl
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 "Load data from echocardiogram.data, skipping the 'name'
    column (11th column)"
6 filename = 'echocardiogram.data'
7
8 "Columns to load: all except the 11th (index 10) which is the
     'name' column"
9 usecols = list(range(10)) + [11, 12]
10 data = np.genfromtxt(filename, delimiter=',', usecols=usecols
    , dtype=float, encoding=None)
11
12 "Prepare input features and target values"
13 x = data[:, :-1].T
14 y_t = data[:, -1].reshape(1, -1)
```

```python

"Min and max values for each feature (before normalization)"
print(np.transpose([np.arange(x.shape[0]), x.min(axis=1), x.max(axis=1)]))

"Normalize features to the range [-1, 1]"
x_min = x.min(axis=1)
x_max = x.max(axis=1)
x_norm_min, x_norm_max = -1, 1
x_norm = (x_norm_max - x_norm_min) / (x_max[:, None] - x_min[:, None]) * (x - x_min[:, None]) + x_norm_min

"Check the range after normalization"
print(np.transpose([np.arange(x_norm.shape[0]), x_norm.min(axis=1), x_norm.max(axis=1)]))

"Sort target values and features"
y_t_s_ind = np.argsort(y_t[0])
x_n_s = x_norm[:, y_t_s_ind]
y_t_s = y_t[:, y_t_s_ind]

"Plot original target values (unsorted)"
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(y_t[0], label='Unsorted')
plt.title('Unsorted Target Labels')
plt.xlabel('Index')
plt.ylabel('Target Value')
plt.grid(True)
plt.legend()

"Plot sorted target values"
plt.subplot(1, 2, 2)
plt.plot(y_t_s[0], label='Sorted', color='orange')
plt.title('Sorted Target Labels')
plt.xlabel('Sorted Index')
plt.ylabel('Target Value')
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.show()

"Save the data to a file using hickle"
hkl.dump([x, y_t, x_norm, x_n_s, y_t_s], 'echocardiogram.hkl')
```

Listing 2: Kod przetwarzający dane do pliku .hkl

Tak prezentują się dane przed i po sortowaniu.



Rysunek 4: Przedstawienie sortowania danych

4 Skrypt programu

Poniżej znajduje się omówienie poszczególnych częci kodu sieci neuronowej.

W pierwszej części inicjalizowane są dane zamieszczone pod kodem oraz biasy i wagi dla poszczególnych warstw.

```
def __init__(self, x, y_t, K1, K2, lr, err_goal, \
                disp_freq, mc, max_epoch, initialize):
        self.x          = x
        self.L          = self.x.shape[0]
        self.y_t        = y_t
        self.K1         = K1
        self.K2         = K2
        self.lr         = lr
        self.err_goal   = err_goal
        self.disp_freq  = disp_freq
        self.mc         = mc
        self.max_epoch  = max_epoch
        self.K3         = y_t.shape[0] if len(y_t.shape)>1 else 1
        self.SSE_vec    = []
        self.PK_vec     = []
        self.data       = self.x.T
        self.target     = self.y_t
        self.initialize = initialize

        if self.initialize:
            self.w1, self.b1 = net.nwtan(self.K1, self.L)
            self.w2, self.b2 = net.nwtan(self.K2, self.K1)
            self.w3, self.b3 = net.rands(self.K3, self.K2)
            hkl.dump([self.w1,self.b1,self.w2,self.b2,self.w3,self.b3], 'wagi3w.hkl')
        else:
            self.w1,self.b1,self.w2,self.b2,self.w3,self.b3 = hkl.load('wagi3w.hkl')

        self.w1_t_1, self.b1_t_1, self.w2_t_1, self.b2_t_1, self.w3_t_1, self.b3_t_1 = \
            self.w1, self.b1, self.w2, self.b2, self.w3, self.b3
        self.SSE = 0
        self.lr_vec = list()
```

Listing 3: Inicjalizacja danych

- **x** - dane wejściowe
- **L** - liczba cech w danych wejściowych

- **y_t** - oczekiwane wyjście (target)
- **K1, K2, K3** - liczba neuronów w kolejnych warstwach sieci
- **lr** - współczynnik uczenia (learning rate)
- **err_goal** - docelowy poziom błędu
- **mc** - współczynnik momentum
- **max_epoch** - maksymalna liczba epok treningowych
- **SSE_vec** - wektor przechowujący błąd sumy kwadratów (SSE)

Funkcja przewiduje wyniki na podstawie wartości wejść. Wyjście pierwszej warstwy jest określane na podstawie wartości wejciowych, każda następna warstwa jest determinowana na podstawie wyjścia poprzedniej warstwy

```
    def predict(self,x):
      n = np.dot(self.w1, x)
      self.y1 = net.tansig( n,  self.b1*np.ones(n.shape))
      n = np.dot(self.w2, self.y1)
      self.y2 = net.tansig( n,  self.b2*np.ones(n.shape))
      n = np.dot(self.w3, self.y2)
      self.y3 = net.purelin(n, self.b3*np.ones(n.shape))
      return self.y3
```

Listing 4: Funkcja predict

Poniżej znajduje się funkcja train wykonująca następujące czynności

- Rozpoczyna pętlę działającą do osiągnięcia maksymalnej liczby epok (2)
- Przewiduje wyjście dla danych wejściowych (3)
- Oblicza błąd (4)
- Zapisuje poprzednią wartość błędu SSE (5)
- Oblicza sumę kwadratów błędów (SSE) (6)
- Oblicza poprawność klasyfikacji (7)
- Dodaje wartość PK do listy (8)
- Sprawdza istnienie warunków zakończenia uczenia (9)
- Sprawdza, czy SSE jest inną wartością niż numer (13)
- Oblicza deltę dla każdej warstwy (16)
- Oblicza gradient dla każdej warstwy (19)
- Aktualizuje wartoć wagi i biasów z momentum każdej warswy (23)
- Dodaje wartość SSE do listy (30)

```
1    def train(self, x_train, y_train):
2        for epoch in range(1, self.max_epoch+1):
3            self.y3 = self.predict(x_train)
4            self.e = y_train - self.y3
5
6            self.SSE_t_1 = self.SSE
7            self.SSE = net.sumsqr(self.e)
8            self.PK = sum((abs(self.e)<0.5).astype(int)[0])/
                self.e.shape[1] * 100
9            self.PK_vec.append(self.PK)
10           if self.SSE < self.err_goal or self.PK == 100:
11               break
12
13           if np.isnan(self.SSE):
14               break
15
16           self.d3 = net.deltalin(self.y3, self.e)
17           self.d2 = net.deltatan(self.y2, self.d3, self.w3)
18           self.d1 = net.deltatan(self.y1, self.d2, self.w2)
19           self.dw1, self.db1 = net.learnbp(x_train,  self.
                d1, self.lr)
```

```
            self.dw2, self.db2 = net.learnbp(self.y1,  self.d2, self.lr)
            self.dw3, self.db3 = net.learnbp(self.y2,  self.d3, self.lr)

            self.w1 += self.dw1 + self.mc * (self.w1 - self.w1_t_1)
            self.b1 += self.db1 + self.mc * (self.b1 - self.b1_t_1)
            self.w2 += self.dw2 + self.mc * (self.w2 - self.w2_t_1)
            self.b2 += self.db2 + self.mc * (self.b2 - self.b2_t_1)
            self.w3 += self.dw3 + self.mc * (self.w3 - self.w3_t_1)
            self.b3 += self.db3 + self.mc * (self.b3 - self.b3_t_1)

            self.SSE_vec.append(self.SSE)
```

Listing 5: Trenowanie sieci

Funkcja train_CV odpowiada za ocenę wydajnoci modelu:

- Wypełnia listę wartosci PK zerami. (2)
- Rozpoczyna pętlę iterującą przez indeksy zbiorów treningowych oraz testowych działającą przez wartoć CVN (4)
- Dzieli na dane treningowe i testowe. (5)
- Dzieli etykiety na treningowe i testowe. (6)
- Trenuje model na danych treningowych. (8)
- Prognozuje dla danych testowych (9)
- Oblicza ilosć próbek testowych (11)
- Oblicza poprawnosc klasyfikacji (12)
- Oblicza srednią wartosć PK (13)
- Zwraca srenią wartosć PK (14)

```
    def train_CV(self, CV, skfold):
      PK_vec = np.zeros(CVN)

      for i, (train, test) in enumerate(skfold.split(self.
          data, np.squeeze(self.target)), start=0):
          x_train, x_test = self.data[train], self.data[
              test]
          y_train, y_test = np.squeeze(self.target)[train],
               np.squeeze(self.target)[test]

          self.train(x_train.T, y_train.T)
          result = self.predict(x_test.T)

          n_test_samples = test.size
          PK_vec[i] = sum((abs(result - y_test)<0.5).astype
              (int)[0])/n_test_samples * 100
      PK = np.mean(PK_vec)
      return PK
```

Listing 6: Ocena wydajnosci modelu

Poniżej fragment odpowiedzialny za przypisanie wartosci zmiennych.

- x - znormalizowane dane wejsciowe
- max_epoch maksymalna liczba epok
- err_goal - docelowy poziom błędu
- disp_freq - częstotlwosć wyswietlanie wyników
- mc_vec wektor współczynników uczenia
- K1_vec, K2_vec - wektory liczb neuronów
- start = timer () - Funkcja włączająca licznik czasu mierzący czas uczenia się sieci.

```
 x,y_t,x_norm,x_n_s,y_t_s = hkl.load('echocardiogram.hkl')

x = x_norm

max_epoch = 200
err_goal = 0.25
disp_freq = 10

lr_vec = np.array([1e-3, 1e-4, 1e-5 ])
mc_vec = np.array([0.01, 0.1, 0.3, 0.5, 0.7, 0.9, 0.99])
K1_vec = np.array([1,3,5,7])
K2_vec = K1_vec

start = timer()
```

Listing 7: Przypisanie wartości zmiennym

Oto opis kolejnej częsic kodu. Ta odpowiada za sprawdzanie optymalnej liczby neuronów w warstwach.

- x - Przypisuje liczbę podziałów w walidacji krzyżowej (1)
- Inicjalizuje walidację krzyżową (2)
- Tworzy wypełnioną zerami macierz wyników dla kombinacji K1 i K2 (3)
- Tworzy wypełnioną zerami macierz wyników dla kombinacji lr i mc (4)
- Tworzy maksymalną wartość PK dla kombinacji K1 i K2 i przypisuje jej 0 (5)

- Tworzy indeks maksymalnej wartości K1 i przypisuje mu 0 (6)
- Tworzy indeks maksymalnej wartości K2 i przypisuje mu 0 (7)
- Inicjuje pętlę w pętli sprawdzającą wszystkie możliwe kombinacje liczby neuronów z wektoru (8)
- Inicjuje sieć neuronową z określonymi K1 i K2 (10)
- Trenuje model z walidacją krzyżową (13)
- Zapisuje wynik PK do macierzy (14)
- Aktualizuje maksymalne PK, gdy bieżące PK jest większe (15)
- Zapisuje indeks K1 dla maksymalnego PK (16)
- Zapisuje indeks K2 dla maksymalnego PK (17)

```
1  CVN = 10
2  skfold = StratifiedKFold(n_splits=CVN)
3  PK_2D_K1K2 = np.zeros([len(K1_vec),len(K2_vec)])
4  PK_2D_lrmc = np.zeros([len(lr_vec),len(mc_vec)])
5  PK_2D_K1K2_max = 0
6  k1_ind_max = 0
7  k2_ind_max = 0
8  for k1_ind in range(len(K1_vec)):
9      for k2_ind in range(len(K2_vec)):
10         mlpnet = mlp_m_3w(x, y_t, K1_vec[k1_ind], K2_vec[k2_
               ind],  \
11                           lr_vec[-1], err_goal, disp_freq,
                                mc_vec[-1], \
12                            max_epoch, True)
13         PK = mlpnet.train_CV(CVN, skfold)
14         print("K1 {} | K2 {} | PK {}". format(K1_vec[k1_ind],
               K2_vec[k2_ind], PK))
15         PK_2D_K1K2[k1_ind, k2_ind] = PK
16         if PK > PK_2D_K1K2_max:
17             PK_2D_K1K2_max = PK
18             k1_ind_max = k1_ind
19             k2_ind_max = k2_ind
```

Listing 8: Sprawdzanie optymalnej liczby neuronów w warstwach K1 i K2

Ostatni omawiany fragment kodu wykonuje poniższe czynnosci:

- Tworzy maksymalną wartość PK dla kombinacji lr i mc i przypisuje jej 0. (1)

- Tworzy indeks maksymalnej wartości lr i przypisuje mu 0. (2)
- Tworzy indeks maksymalnej wartości mc i przypisuje mu 0. (3)
- Inicjuje iterację w iteracji sprawdzającą wszystkie możliwe kombinacje współczynników lr i mc z wektorów. (5-6)
- Inicjuje sieć neuronową z optymalnymi wartościami K1 i K2. (7)
- Ustawia bieżące wartości współczynników lr oraz mc. (8)
- Trenuje model z walidacją krzyżową. (10)
- Zapisuje wynik PK do macierzy. (12)
- Aktualizuje maksymalną PK, gdy bieżące PK jest większe. (13)
- Zapisuje indeks lr dla maksymalnego PK. (15)
- Zapisuje indeks mc dla maksymalnego PK. (16)

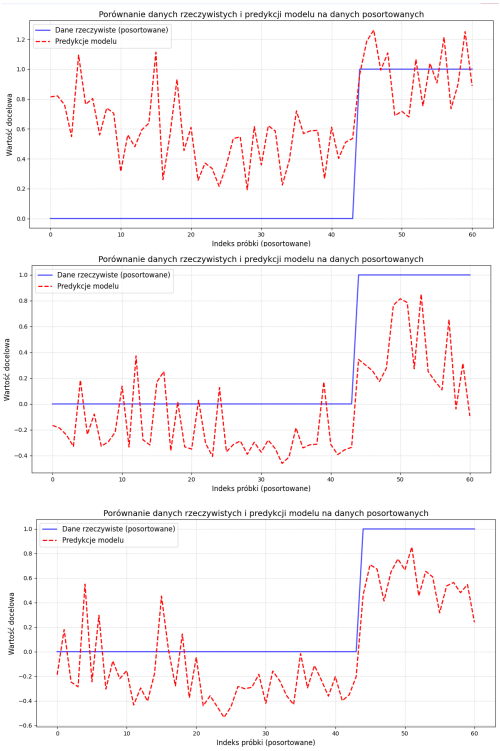
```
1  PK_2D_lrmc_max = 0
2  lr_ind_max = 0
3  mc_ind_max = 0
4
5  for lr_ind in range(len(lr_vec)):
6      for mc_ind in range(len(mc_vec)):
7          mlpnet = mlp_m_3w(x, y_t, K1_vec[k1_ind_max], K2_vec[
               k2_ind_max],  \
8                                lr_vec[lr_ind], err_goal, disp_
                                    freq, mc_vec[mc_ind], \
9                                 max_epoch, True)
10         PK = mlpnet.train_CV(CVN, skfold)
11         print("lr \{} | mc {} | PK {}".format(lr_vec[lr_ind],
               mc_vec[mc_ind], PK))
12         PK_2D_lrmc[lr_ind, mc_ind] = PK
13         if PK > PK_2D_lrmc_max:
14             PK_2D_lrmc_max = PK
15             lr_ind_max = lr_ind
16             mc_ind_max = mc_ind
```

Listing 9: Sprawdzanie optymalnej liczby neuronów w warstwach K1 i K2

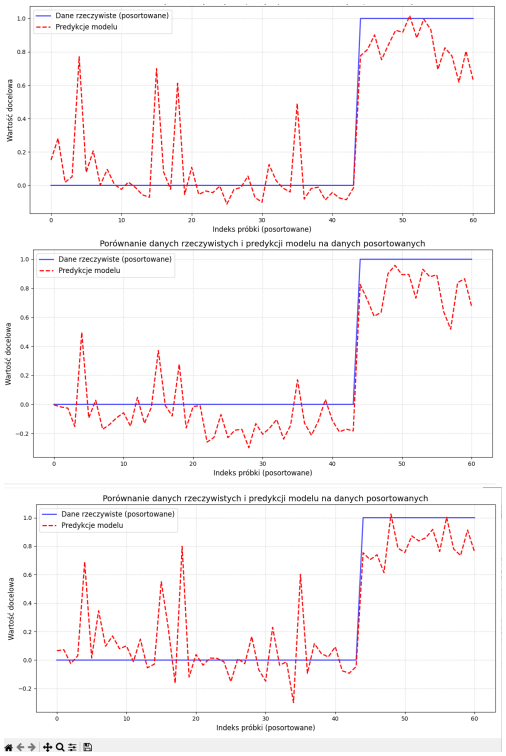
5 Eksperymenty

5.1 Sprawdzenie wpływu ilosci epok na dokładnosć modelu

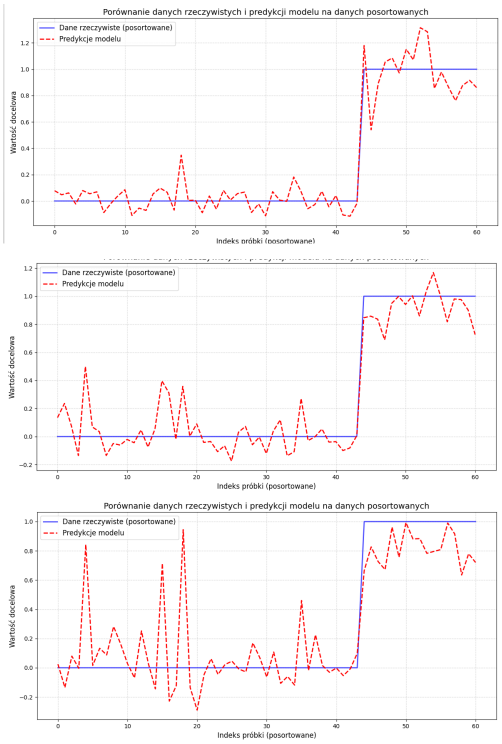
Ponieważ model widzi jedynie x_train, a x_test jest używane tylko do oceny jakosci przy treningu, wydaje się dosć odporny na przeuczenie. Postanowiłem więc sprawdzić wpływ ilosci epok na dane, które przewidzi model. Przy sprawdzaniu wyników użyłem zakresu [1,3,5,7,9,11,13,15] dla obu neuronów.Przetestowałem kolejno 10, 200 i 10 000 epok dokonując trzech pomiarów dla każdej z powyższych ilosci.



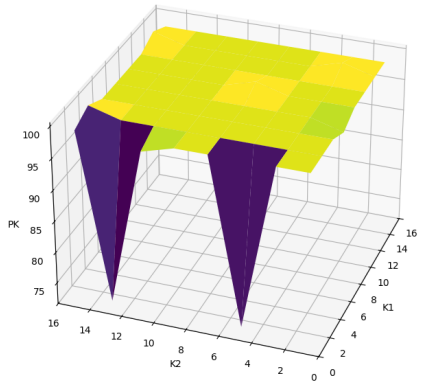
Rysunek 5: Porównanie danych rzeczywistych i predykcji modelu przy uczeniu trwającym 10 epok



Rysunek 6: Porównanie danych rzeczywistych i predykcji modelu przy uczeniu trwającym 200 epok



Rysunek 7: Porównanie danych rzeczywistych i predykcji modelu przy uczeniu trwającym 10 000 epok



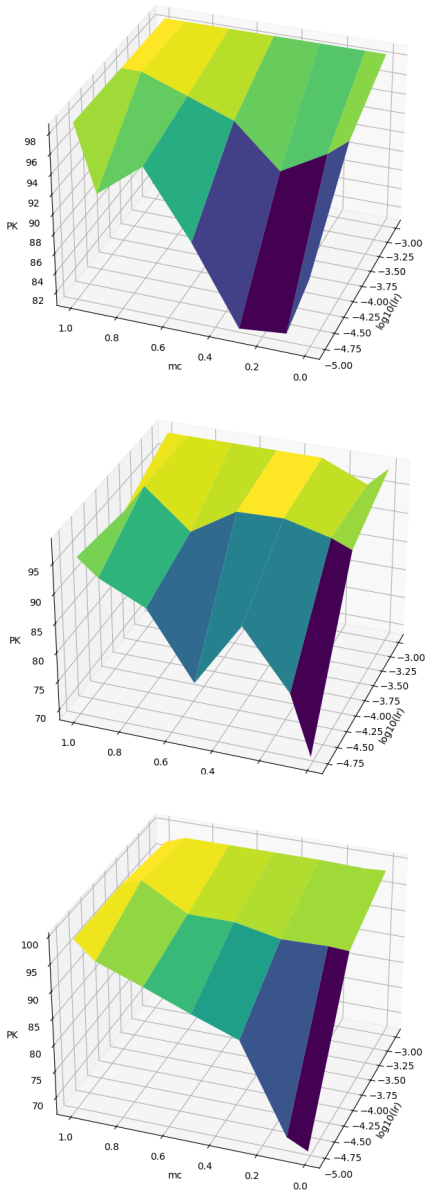
Rysunek 8: Zależność ilosci neuronów od dokładnosci przy uczeniu trwajacym 10 000 epok.

W przypadku treningu modelu przez 10 epok zaobserwowano wyraźne niedouczenie. Zwiększenie liczby epok do 200 pozwoliło na istotne zmniejszenie różnicy pomiędzy przewidywaniami modelu a wartościami rzeczywistymi. Dalsze zwiększanie liczby epok, aż do 10 000, nie prowadziło do zauważalnej poprawy dokładności predykcji, natomiast znacząco wydłużało czas obliczeń. Jednocześnie zaobserwowano, że przy większej liczbie epok wpływ doboru parametrów K_1 i K_2 na wyniki modelu ulegał osłabieniu — różne wartości tych parametrów prowadziły do zbliżonych rezultatów.

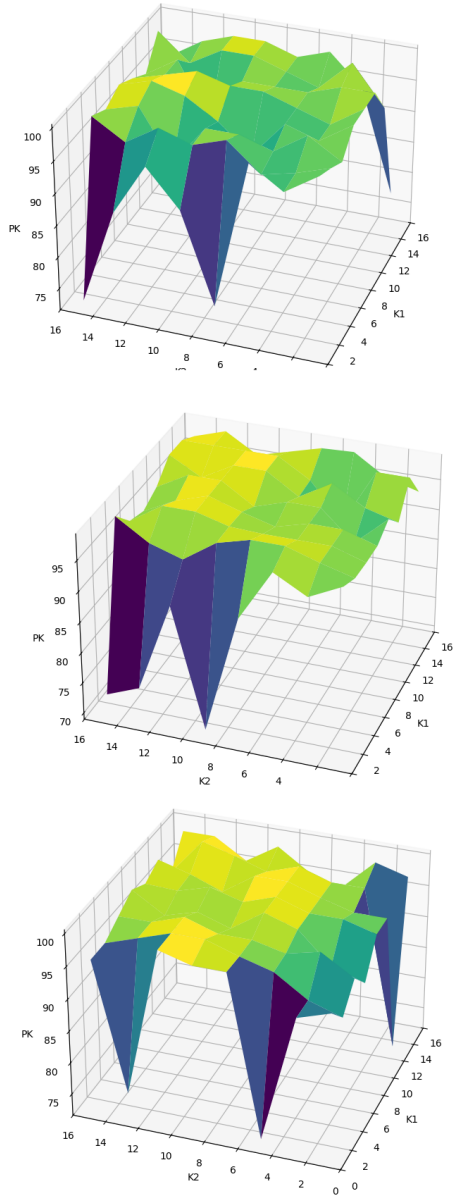
5.2 Wyznaczanie optymalnych wartosci K1, K2, lr oraz mc

Podczas eksperymentu zbadano wpływ ilosci neuronów poszczególnej warstwy na poprawnosć klasyfikacji. Zbadano również wpływ wartosci przypieszenia współczynnika uczenia oraz wartosci współczynnika momentum. użyto zakresu [1,3,5,7,9,11,13,15] dla obu neuronów, zakresu [1e-3, 1e-4, 1e-5] dla

wartosci przyspieszenia współczynnika uczenia, zakresu [0.01, 0.1, 0.3, 0.5, 0.7, 0.99] dla wartosci współczynnika momentum oraz 200 epok. Wykonane zostały 3 powtórzenia pomiarów.



Rysunek 9: mc/log10(lr)/PK



Rysunek 10: K1/K2/PK

Pomiar 1: K1=3 | K2=5 | lr=0.001 | mc=0.01 | PK=98.57142857142858
Pomiar 2: K1=7 | K2=13 | lr=0.001 | mc=0.01 | PK=98.57142857142858
Pomiar 3: K1=13 | K2=13 | lr=0.0001 | mc=0.9 | PK=100.0

Obserwacje pokazują nam, że zarówno wyższa wartoć współczynnika momentum i niższa wartoć współczynnika uczenia są skorelowane z dokładniejszymi pomiarami. Jednoczesnie wyjątkiem od tej reguły jest najniższa wartosc mc, która ma nieproporcjonalnie wyższą dokładnosć od innych wartosci. Sprawa wygląda mniej jasno w przypadku ilosci neuronów. Co prawda liczba 13 była najoptymalniejsza dla K2 aż dwa razy i raz dla K1, ale nie możemy tu mówić o regule. Wartosc 13,13 w jednym pomiarze była najlepszym wynikiem, a w innym jednym z najgorszych. Punkty z wysoką dokładnoscią zdają się pojawiać w dużej mierze losowo. Nie mniej jednak okolice 13 neuronów na warstwie K2 zdają się w jakis sposób korelować z wyższym PK. W przypadku warstwy K1 tudno dopatrzyć się zależnosci.

6 Podsumowanie i wnioski

W przeprowadzonych eksperymentach zbadano wpływ parametrów funkcji uczącej na dokładność modelu. Wykazano, że zwiększenie liczby epok prowadzi do poprawy jakości estymacji jedynie do pewnego momentu, przy czym pozytywnie wpływa na wyniki dla różnych konfiguracji liczby neuronów. Dla 200 epok jedynie nieliczne kombinacje liczby neuronów prowadziły do wyników zbliżonych do 100% poprawności. Natomiast przy 10 000 epok niemal wszystkie konfiguracje osiągały bardzo wysoką dokładność. Tak długi trening znacząco obciążał obliczeniowo model, jednak nie zaobserwowano oznak przeuczenia.

Nie udało się jednoznacznie ustalić zależnosci pomiędzy konkretną liczbą neuronów a poprawą dokładności. Uzyskane wyniki wykazywały znaczną zmienność, co sugeruje, że do uchwycenia ewentualnych zależności wymagane byłyby znacznie większe próby. Konfiguracje neuronów, w których najczęściej uzyskiwano wysoką dokładność, miały zazwyczaj około 13 neuronów w warstwie K_2, jednak należy brać pod uwagę możliwość przypadkowości tej obserwacji. Eksperymenty nie wykazały istotnego wpływu liczby neuronów w warstwie K_1 na końcową jakość estymacji.

Analiza wpływu parametrów lr (learning rate) oraz mc (momentum coefficient) wykazała, że model zwykle osiągał wyższą poprawność dla większych wartości tych parametrów. We wszystkich testach konfiguracja, w której oba parametry miały najwyższe wartości, wyróżniała się wyjątkowo wysoką dokładnością. Jednocześnie konfiguracje z bardzo małym mc również cechowały

się dobrymi wynikami, co sugeruje, że niski współczynnik momentum nie musi negatywnie wpływać na skuteczność modelu.